

CO215 Advanced Programming Laboratory

Spring Semester 2026

Assignment	Assignment 8 – Course Enrollment Dashboard in ReactJS
Roll Number	CSB24073
Name	Gaurav Nath
Session / Year	Spring / 2026

1. Aim

To develop a Course Enrollment Dashboard as a React functional component that maintains student data in state, supports adding and removing students, displays students sorted by GPA, lists all unique courses, and filters students by course — using React hooks, Map, Set, and functional array methods.

2. Problem Description

Each student is represented as an object with the following structure:

```
{
  id: number,
  name: string,
  enrolledCourses: Set<string>,
  gpa: number
}
```

The dashboard must implement all of the following features:

- Add a new student with name, courses (comma-separated), and GPA.
- Remove a student by their ID.
- Display all students sorted by GPA in descending order.
- Display all unique courses across all students.
- Filter and display only students enrolled in a selected course.

3. Requirements

- Use `useState` to maintain the student list in React state.
- Use a Map internally for efficient id-to-student mapping.
- Use Set for course uniqueness — both per-student and globally.
- Use `map`, `filter`, and `reduce` for derived data computation.
- Never mutate state directly — always create new Map/Set/object copies.
- Use the spread operator (`...`) for all state updates.
- Convert Set to array (using `[...set]`) before rendering in JSX.

4. Algorithm

4.1 State Initialization

- Initialize a Map with 4 pre-loaded students, each having a Set of courses.
- `useState` wraps the Map so React can track changes.

4.2 Add Student

- Read name, courses string, and GPA from input fields.
- Generate a new ID using `Math.max(...studentMap.keys()) + 1`.
- Convert the comma-separated course string into a Set.
- Create a new student object and spread the existing Map into a new Map with the addition.
- Call `setStudentMap` with the new Map — no direct mutation.

4.3 Remove Student

- Filter the Map entries to exclude the target ID.
- Reconstruct a new Map from the filtered entries and call `setStudentMap`.

4.4 Sort by GPA

- Spread Map values into an array: `[...studentMap.values()]`
- Sort using `.sort((a, b) => b.gpa - a.gpa)` for descending order.

4.5 Unique Courses

- Use `reduce()` over all students, adding each course to an accumulator Set.
- Convert to array with `[...allCourses]` before rendering as JSX elements.

4.6 Filter by Course

- Use `.filter(s => s.enrolledCourses.has(course))` on the sorted array.
- `Set.has()` provides $O(1)$ lookup per student.

5. ReactJS Implementation

File: `CourseEnrollmentDashboard.jsx`

```
import { useState } from "react";

// Initial student data stored in a Map (id -> student)
const initialStudentMap = new Map([
  [1, { id: 1, name: "Rahul", enrolledCourses: new Set(["Math", "DSA"]),
    gpa: 8.5 }],
  [2, { id: 2, name: "Anita", enrolledCourses: new Set(["Math", "Java"]),
    gpa: 9.1 }],
  [3, { id: 3, name: "Karan", enrolledCourses: new Set(["DSA", "Java", "OS"]),
    gpa: 7.8 }],
  [4, { id: 4, name: "Priya", enrolledCourses: new Set(["OS", "Networks"]),
    gpa: 8.9 }],
]);

export default function CourseEnrollmentDashboard() {

  // — State —
  const [studentMap, setStudentMap] = useState(initialStudentMap);
  const [filterCourse, setFilterCourse] = useState("");
  const [newName, setNewName] = useState("");
  const [newCourses, setNewCourses] = useState("");
  const [newGpa, setNewGpa] = useState("");

  // — Derived: sorted by GPA descending —
  const sortedStudents = [...studentMap.values()].sort(
    (a, b) => b.gpa - a.gpa
  );

  // — Derived: use .map() to build display-ready student objects
```

```
// map() transforms each student into a display object with courses as
array
const displayStudents = sortedStudents.map(s => ({
  ...s,
  coursesArray: [...s.enrolledCourses], // Set -> Array for rendering
  courseCount: s.enrolledCourses.size,
}));

// — Derived: all unique courses via reduce + Set —————
const allCourses = [...studentMap.values()].reduce((acc, student) => {
  [...student.enrolledCourses].forEach(c => acc.add(c));
  return acc;
}, new Set());

// — Derived: filter students by selected course —————
const filteredStudents = filterCourse
  ? sortedStudents.filter(s => s.enrolledCourses.has(filterCourse))
  : sortedStudents;

// — Add new student (no direct mutation, spread operator) —————
const handleAddStudent = () => {
  if (!newName.trim() || !newCourses.trim() || !newGpa) return;
  const newId = Math.max(...studentMap.keys(), 0) + 1;
  const coursesSet = new Set(
    newCourses.split(",").map(c => c.trim()).filter(Boolean)
  );
  const newStudent = {
    id: newId, name: newName.trim(),
    enrolledCourses: coursesSet, gpa: parseFloat(newGpa),
  };
  // Spread into new Map — no direct mutation
  setStudentMap(new Map([...studentMap, [newId, newStudent]]));
  setNewName(""); setNewCourses(""); setNewGpa("");
};

// — Remove student by ID —————
const handleRemove = (id) => {
  setStudentMap(
    new Map([...studentMap].filter(([key]) => key !== id))
  );
};

return (
  <div>
    <h2>Course Enrollment Dashboard</h2>

    {/* Add Student Form */}
    <section>
      <input placeholder="Name" value={newName}
        onChange={e => setNewName(e.target.value)} />
      <input placeholder="Courses (comma separated)" value={newCourses}
        onChange={e => setNewCourses(e.target.value)} />
      <input placeholder="GPA" type="number" value={newGpa}
        onChange={e => setNewGpa(e.target.value)} />
      <button onClick={handleAddStudent}>Add</button>
    </section>

    {/* All Unique Courses — Set converted to array before rendering */}
    <section>
      <h3>All Unique Courses</h3>
      {[...allCourses].sort().map(course => (
        <span key={course}>{course}</span>
      ))}
    </section>
  </div>
);
```

```

    { /* Filter by Course */
    <section>
      <select value={filterCourse}
        onChange={e => setFilterCourse(e.target.value)}>
        <option value="">-- All Students --</option>
        {[...allCourses].sort().map(c => (
          <option key={c} value={c}>{c}</option>
        ))}
      </select>
    </section>

    { /* Student Table sorted by GPA */
    <table>
      <thead>
        <tr><th>ID</th><th>Name</th><th>Courses</th>
          <th>GPA</th><th>Action</th></tr>
      </thead>
      <tbody>
        {filteredStudents.map(s => (
          <tr key={s.id}>
            <td>{s.id}</td>
            <td>{s.name}</td>
            { /* Convert Set to array before rendering */
            <td>{[...s.enrolledCourses].join(", ")}</td>
            <td>{s.gpa.toFixed(1)}</td>
            <td><button onClick={() => handleRemove(s.id)}>
              Remove</button></td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);
}

```

6. Input

The component is initialized with 4 pre-loaded students in the Map:

ID 1 – Rahul	Courses: Math, DSA	GPA: 8.5
ID 2 – Anita	Courses: Math, Java	GPA: 9.1
ID 3 – Karan	Courses: DSA, Java, OS	GPA: 7.8
ID 4 – Priya	Courses: OS, Networks	GPA: 8.9

Additional students can be added at runtime via the input form (Name, Courses, GPA fields).

7. Output

7.1 Expected Output (Sorted by GPA, Descending)

ID	Name	Courses	GPA
2	Anita	Math, Java	9.1
4	Priya	OS, Networks	8.9
1	Rahul	Math, DSA	8.5
3	Karan	DSA, Java, OS	7.8

All Unique Courses: DSA, Java, Math, Networks, OS

7.2 Screenshot

Course Enrollment Dashboard

Add New Student

All Unique Courses

Filter by Course



Students (Sorted by GPA — Descending)

ID	Name	Enrolled Courses	No. of Courses	GPA	Action
2	Anita	Math, Java	2	9.1	Remove
4	Priya	OS, Networks	2	8.9	Remove
1	Rahul	Math, DSA	2	8.5	Remove
3	Karan	DSA, Java, OS	3	7.8	Remove

8. Time Complexity of Filtering Students by Course

Let n = number of students. `Set.has()` lookup is $O(1)$ on average.

```
// filterStudents by course —  $O(n * m)$ 
// n = number of students
// m = average number of courses per student (Set.has is  $O(1)$  average)

const filteredStudents = sortedStudents.filter(
  s => s.enrolledCourses.has(filterCourse) //  $O(1)$  per student
);
// Total:  $O(n)$  — iterates all students once

// Note: sortedStudents is pre-sorted, so filtering does NOT re-sort.
// Sorting itself costs  $O(n \log n)$  and is done once as derived state.
```

- `filter()` iterates over all n students once: $O(n)$.
- `Set.has(filterCourse)` inside each iteration: $O(1)$ average.
- Overall time complexity of filtering: $O(n)$.
- Pre-sort (once, as derived state) costs $O(n \log n)$ and is not repeated on each filter.

Final Complexity: Filtering — $O(n)$ | Sorting — $O(n \log n)$ | Overall — $O(n \log n)$

9. Learning Outcomes

- Understood how to use `useState` with complex data structures (Map, Set) in React without directly mutating state.
- Practiced the spread operator (...) to create new Map and Set copies for immutable state updates.
- Applied `reduce()` to aggregate unique courses across all students into a single Set.
- Used `filter()` with `Set.has()` for efficient $O(n)$ course-based filtering.
- Learned to convert Set to array using [...set] before rendering JSX, since Sets are not directly iterable in React.
- Reinforced the separation of concerns in React: state initialization, derived computation, and event handlers.